

Data Structures

Lecture 5

Stack II

Abstract Data Type

References:

Data Structures and Algorithms in Java

Michael T. Goodrich and Roberto Tamassia.



University of Kufa
Faculty of Computer Science
and Mathematics

Dr. Munqath Al-Attar

Information Technology Research and
Development Center ITRDC

Email:

munqith.alattar@uokufa.edu.iq

Website:

<http://staff.uokufa.edu.iq/profile.html?munqith.alattar>

Topics

- Register stack and Memory stack
- Stack Advantages and Limitations
- Stack Applications
 - Reversing an Array Using a Stack
 - Matching Parentheses
 - Matching Tags in an HTML Document

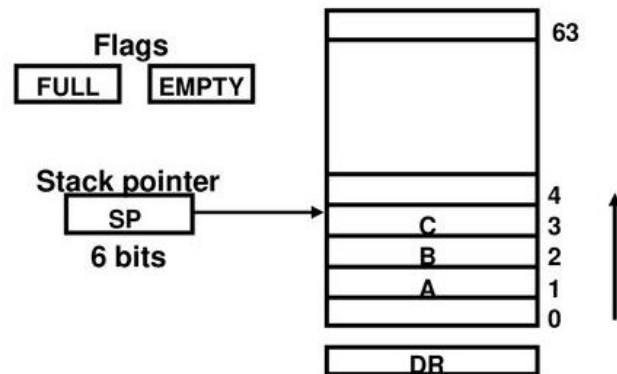


Data Structures

Register Stack

Register Stack

- It is also a memory element present in the memory. It handles only a small amount of data compared to the memory stack, because the stack size is always limited in the register stack.
- A stack can be organized as a collection of finite number of registers that are used to store temporary information during the execution of a program.
- That stack pointer **SP** is a register that holds the address of the top of element of the stack and it always points at the top item in the stack.
- Here in the given picture, **SP** = 3 (address)
- A,B, and C are three items in the stack.
- In 64-word stack, SP accommodate only 6 least significant bits.



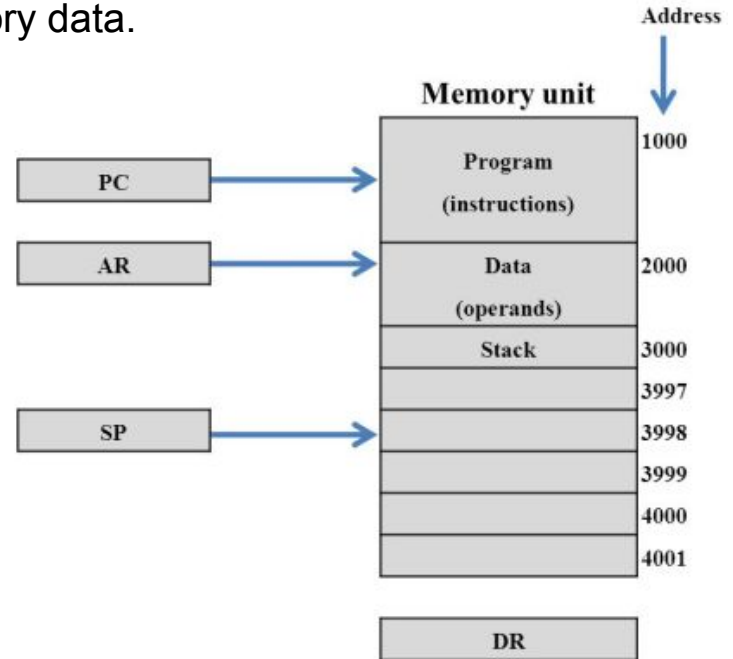
Data Structures

Memory Stack

Memory Stack

The stack size is flexible. It handles a large amount of memory data.

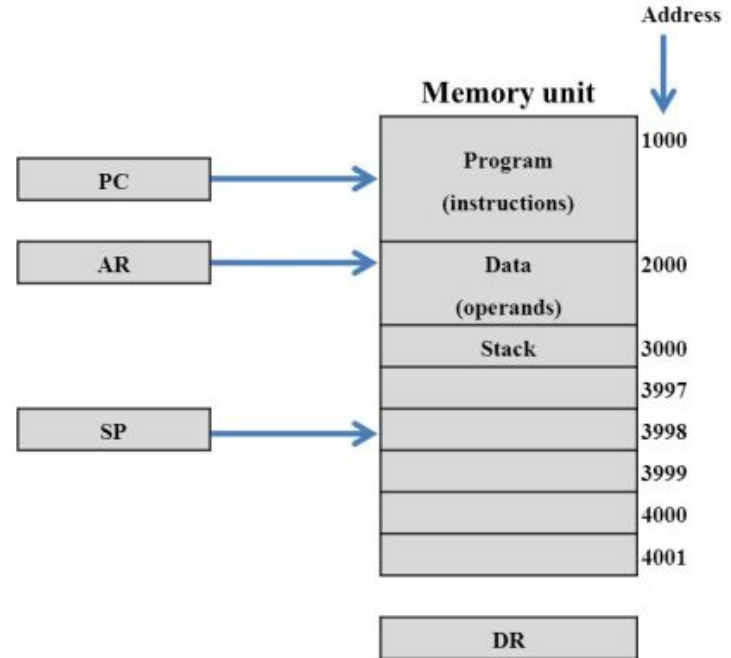
- A stack can be implemented in a random access memory (RAM) attached to a CPU
- The implementation of a stack in the CPU is done by assigning a portion of memory to a stack operation and using a processor register as a stack pointer



Data Structures

Memory Stack

- The starting memory location of the stack is specified by the processor register as a stack pointer **SP**.
- Here the, portion of computer memory are
 - Program
 - Data
 - Stack
- **PC** is used during the fetch phase to read an instruction.
- **AR** is used during the execution phase to read an operand.
- The advantage of memory stack id that the CPU can refer to it without having to specify an address, since the address is always available and automatically updated in the stack pointer.



Data Structures

Memory Stack

Register Stack	Memory Stack
Momentary spaces for internal processors	Series of memory spaces used in the processes that are done by the processor and temporarily stored in registers
Generally on CPU	On RAM
Faster access to registers	Slower access to memory
Limited in Size	LArger in size

Data Structures

Stack Advantages and Limitations

Advantages of Stack:

- Useful in managing data that in LIFO order.
- Used for systematic Memory Management.
- Used in many virtual machines like Java virtual machine (JVM) .
- Efficient function management:
When a function is called, the local variables are stored in the stack and automatically destroyed once returned from the function.
- Stack allows control over memory allocation and deallocation.
- Stack cleans up the objects automatically.



Data Structures

Stack Advantages and Limitations

Disadvantages of Stack:

- Limited size.
- Stack size must be defined before.
- Stack overflow.
- Random accessing is not possible.

Data Structures

Stack Applications

Reversing an Array Using a Stack

- We can use a stack to reverse the elements in an array, thereby producing a nonrecursive algorithm for the array-reversal problem
- The basic idea is simply to push all the elements of the array in order into a stack and then fill the array back up again by popping the elements off of the stack.

A generic method that reverses the elements in an array using a stack declared using the Stack interface.

```
/** A nonrecursive generic method for reversing an array */  
public static <E> void reverse(E[] a) {  
    Stack<E> S = new ArrayStack<E>(a.length);  
    for (int i=0; i < a.length; i++)  
        S.push(a[i]);  
    for (int i=0; i < a.length; i++)  
        a[i] = S.pop();  
}
```

Data Structures

Stack Applications

```
/** Tester routine for reversing arrays */
public static void main(String args[]) {
    Integer[] a = {4, 8, 15, 16, 23, 42}; // autoboxing allows this
    String[] s = {"Jack", "Kate", "Hurley", "Jin", "Boone"};
    System.out.println("a = " + Arrays.toString(a));
    System.out.println("s = " + Arrays.toString(s));
    System.out.println("Reversing...");
    reverse(a);
    reverse(s);
    System.out.println("a = " + Arrays.toString(a));
    System.out.println("s = " + Arrays.toString(s));
}
```

```
/** A nonrecursive generic method for reversing an array */
public static <E> void reverse(E[] a) {
    Stack<E> S = new ArrayStack<E>(a.length);
    for (int i=0; i < a.length; i++)
        S.push(a[i]);
    for (int i=0; i < a.length; i++)
        a[i] = S.pop();
}
```



Data Structures

Stack Applications

```
/** Tester routine for reversing arrays */  
public static void main(String args[]) {  
    Integer[] a = {4, 8, 15, 16, 23, 42}; // autoboxing allows this  
    String[] s = {"Jack", "Kate", "Hurley", "Jin", "Boone"};  
    System.out.println("a = " + Arrays.toString(a));  
    System.out.println("s = " + Arrays.toString(s));  
    System.out.println("Reversing. . .");  
    reverse(a);  
    reverse(s);  
    System.out.println("a = " + Arrays.toString(a));  
    System.out.println("s = " + Arrays.toString(s));  
}
```

The output from this method is the following:

```
a = [4, 8, 15, 16, 23, 42]  
s = [Jack, Kate, Hurley, Jin, Michael]  
Reversing...  
a = [42, 23, 16, 15, 8, 4]  
s = [Michael, Jin, Hurley, Kate, Jack]
```



Data Structures

Stack Applications

Matching Parentheses and HTML Tags

Arithmetic expressions can contain various pairs of grouping symbols, such as

- Parentheses: "(" and ")"
- Braces: "{" and "}"
- Brackets: "[" and "]"
- ...

and each opening symbol must match with its corresponding closing symbol. For example, a left bracket, "[," must match with a corresponding right bracket, "]," as in the following expression: $[(5 + x) - (y + z)]$.



Data Structures

Stack Applications

The following examples further illustrate this concept:

- Correct: ()(()){([())}}
- Correct: ((()(()){([())}}))
- Incorrect:)(()){([())}}
- Incorrect: ({[]})}
- Incorrect: (

We can use a stack *S* to perform the matching of grouping symbols in an arithmetic expression with a single left-to right scan. The algorithm tests that left and right symbols match up and also that the left and right symbols are both of the same type.



Data Structures

Stack Applications

An Algorithm for Parentheses Matching

- Suppose we are given a sequence $X = x_0 x_1 x_2 \dots x_{n-1}$, where each x_i is a token that can be a grouping symbol, a variable name, an arithmetic operator, or a number.
- Each time we encounter an opening symbol, we push that symbol onto S , and each time we encounter a closing symbol, we pop the top symbol from the stack S (assuming S is not empty) and we check that these two symbols are of the same type.
- If the stack is empty after we have processed the whole sequence, then the symbols in X match.
- Assuming that the push and pop operations are implemented to run in constant time, this algorithm runs in $O(n)$, that is linear, time.



Data Structures

Stack Applications

Algorithm ParenMatch(X, n):

Input: An array X of n tokens, each of which is either a grouping symbol, a variable, an arithmetic operator, or a number

Output: **true** if and only if all the grouping symbols in X match

Let S be an empty stack

for $i \leftarrow 0$ to $n - 1$ **do**

if $X[i]$ is an opening grouping symbol **then**

$S.push(X[i])$

else if $X[i]$ is a closing grouping symbol **then**

if $S.isEmpty()$ **then**

return false {nothing to match with}

if $S.pop()$ does not match the type of $X[i]$ **then**

return false {wrong type}

if $S.isEmpty()$ **then**

return true {every symbol matched}

else

return false {some symbols were never matched}

Data Structures

Stack Applications

Matching Tags in an HTML Document

- HTML is the standard format for hyperlinked documents on the Internet.
- In an HTML document, portions of text are delimited by HTML tags.
- Commonly used HTML tags include
 - body: document body
 - h1: section header
 - center: center justify
 - p: paragraph
 - ol: numbered (ordered) list
 - li: list item.



Data Structures

Stack Applications

See Code Fragment 5.12: in page 283

```
<body>
<center>
<h1> The Little Boat </h1>
</center>
<p> The storm tossed the little
boat like a cheap sneaker in an
old washing machine. The three
drunken fishermen were used to
such treatment, of course, but
not the tree salesman, who even as
a stowaway now felt that he
had overpaid for the voyage. </p>
<ol>
<li> Will the salesman die? </li>
<li> What color is the boat? </li>
<li> And what about Naomi? </li>
</ol>
</body>
```

(a)

The Little Boat

The storm tossed the little boat like a cheap sneaker in an old washing machine. The three drunken fishermen were used to such treatment, of course, but not the tree salesman, who even as a stowaway now felt that he had overpaid for the voyage.

1. Will the salesman die?
2. What color is the boat?
3. And what about Naomi?

(b)

